Web Development Foundations

Week 9 – "Modular Thinking"

Day 1: "Breaking Systems Into Parts"

1

Breaking Systems Into Parts

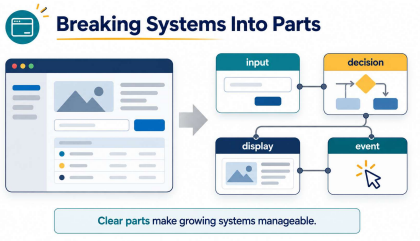
Working code can still become difficult code.

As a program grows, we need to know:

- where each job belongs
- which part should change
- what must stay working
- how the parts connect

Modular thinking makes a larger system understandable.

2



Breaking Systems Into Parts

3

Lab Review: Successful Path

One successful path used small functions to keep the timing explainable:

- an action starts now
- waiting is visible
- a result appears later
- the existing planner still works

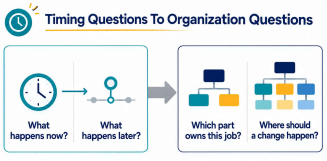


4



Connection: Async Feature Adds A "Responsibility"

5



Week 8 asked:

- What happens now?
- What happens later?

Week 9 adds:

- Which part owns this job?
- Where should a future change happen?
- Can I explain the structure without tracing every line?

6

Modularity And Modules

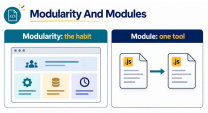
Modularity is the habit:

Break a larger system into understandable parts with clear jobs.

A module is one possible tool:

Place a related piece of code in a file or code unit that another part can use.

You can practice modularity *before* using multiple files.

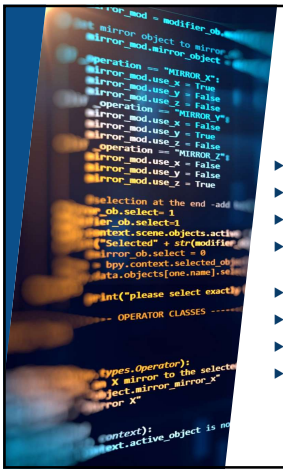


7

What Counts as Success Today

- identify separate responsibilities in working code
- give each responsibility a clear function name
- connect the functions through a simple coordinator
- reorganize without changing the result
- explain why the new structure is easier to follow

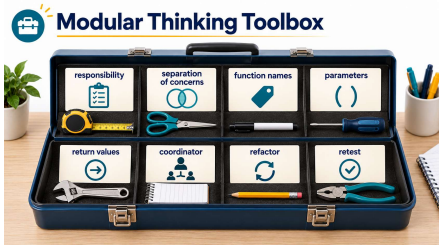
8



What We Will Use Today

- responsibility
- separation of concerns
- clear function names
- parameters and return values
- coordinator function
- event handler
- small-step refactoring
- retesting

9



Modular Thinking Toolbox

10

What We Will "Skip" For Now

Parked for later:

- bundlers and build tools
- npm package workflows
- framework architectures
- advanced `import` / `export`
- dependency injection
- the "perfect" folder structure



For now:

Find the responsibilities. Name them clearly. Keep the program working.

11



Parked Professional Tooling

12



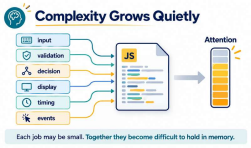
13

Complexity Grows Quietly

A script becomes harder to manage when one place must:

- read user input
- validate or decide
- calculate a result
- update the page
- handle timing
- respond to events

Each job may be small. Together they create cognitive load.



14

Separation Of Concerns

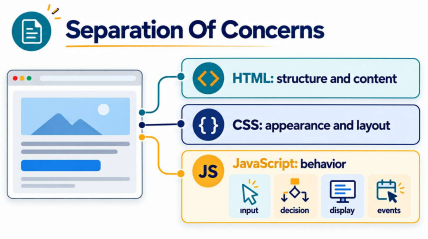
Separation of concerns means giving different jobs clear homes.

In a beginner web project:

- HTML provides structure and content
- CSS controls appearance and layout
- JavaScript controls behavior

Inside JavaScript, responsibilities can also be separated.

15



Separation Of Concerns

16

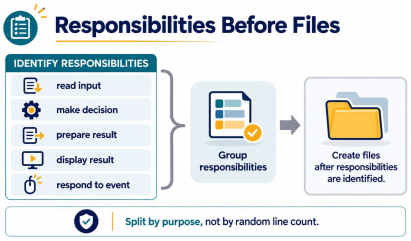
Responsibilities Come Before Files

Before creating more files, identify the jobs:

- read input
- make a decision
- prepare a result
- display the result
- respond to the user's action

Split by purpose, not by random line count.

17



Responsibilities Before Files

18

Functions Give Responsibilities Names

Let's look at functions to get an idea of "HOW" to separate concepts/ideas:

A clear function name answers:
"What job does this part perform?"

Examples:

- `readMinutes()` gets input
- `choosePlan(minutes)` makes a decision
- `updateResult(message)` changes the page

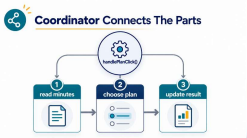
Names reduce the amount of code we must hold in memory at once.

19



Functions As Named Jobs

20



A Coordinator Connects The Parts

A "*coordinator*" function tells the story of the feature:

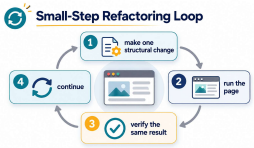
1. read the minutes
2. choose a plan
3. update the result

`handlePlanClick()` connects the parts.

It coordinates the work instead of containing every detail.

21

Refactoring Preserves Behavior



Refactoring changes the structure of working code.

The visible behavior should stay the same.

Safe rhythm:

1. make one small structural change
2. run the page
3. verify the same result
4. continue only after it works

22

Demo #1

"One File, Clear Responsibilities"

23



What to Watch For

- the original behavior works first
- one responsibility moves at a time
- each function receives a purposeful name
- `handlePlanClick()` coordinates the feature
- the browser is retested after each change

The result should behave exactly as it did before.

24

One File Responsibilities Demo

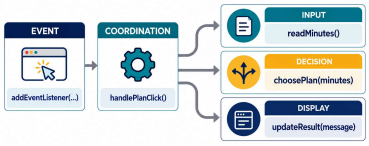


Structure changes. Behavior stays the same.

Demo: One File Responsibilities

25

Responsibility Map



The finished script now has a responsibility map:

- input: `readMinutes()`
- decision: `choosePlan(minutes)`
- display: `updateResult(message)`
- coordination: `handlePlanClick()`
- event connection: `addEventListener(...)`

Each part has a reason to exist.

26

Full Application: Modularity Scaffold

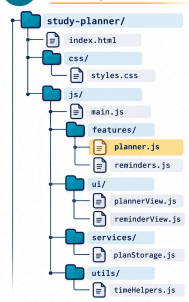
A ***common*** Modular responsibility map for an application:

- **Entry point:** starts the application and connects its features.
- **Features:** organize planner and reminder behavior.
- **UI:** reads controls and updates the page.
- **Services:** handle persistence or external data.
- **Utilities:** provide small, reusable operations.

Each section has a reason to exist.

This concept is a typical use of "modularity thinking" at the application level

Study Planner



27

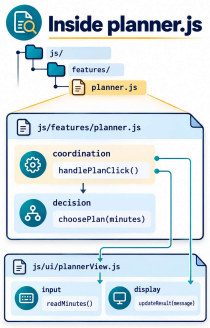
Zooming in Planner.js

As we zoom into one of the files, the same ideas of responsibilities are used:

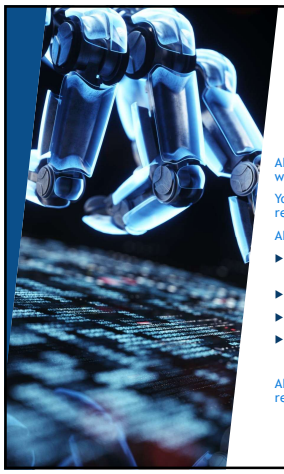
- A coordinator function (like "main")
- Individualized separation of concerns ("functions")
- Decisions, Input and UI (connections to other modules)

Notice that the same idea is now "replicated" at a more detailed view.

Inside planner.js



28



AI Role: Assistant

AI as Assistant ***actively*** begins this week.

You must manually write or revise relevant code first.

AI may help you:

- identify possible responsibility groups
- compare two organization choices
- suggest clearer function names
- explain a small structural tradeoff

AI may not start the assignment or rewrite the whole project.

29

Useful AI Prompt Pattern

"I manually wrote this JavaScript first.

It currently [describe what works].

The code feels difficult to follow because [specific concern].

Do not rewrite the project.

Identify the responsibilities you see and suggest two possible function

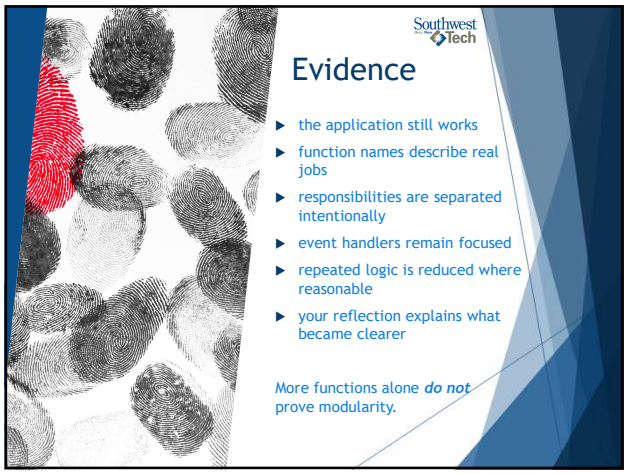
boundaries. Explain why each boundary may help.

Ask any clarifying questions before suggesting code."

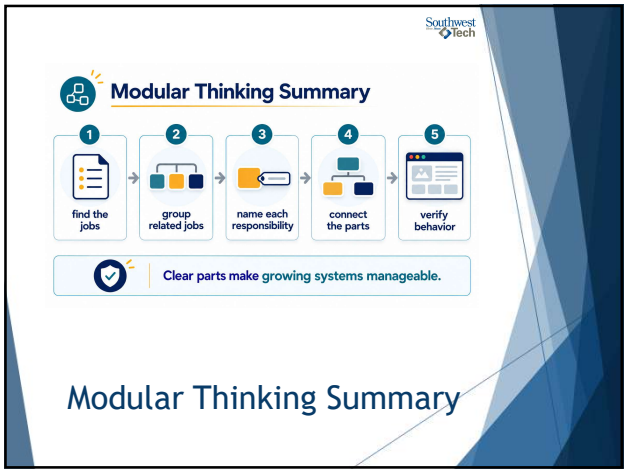
30


Southwest Tech
Lab 09
In Lab 09 - Restructure existing JavaScript into clearer parts.
Your first iteration should include:
at least 3-4 purposeful functions
separate event, data or decision, and display responsibilities
less complexity inside event listeners
the same working behavior after reorganization
Multiple JavaScript files are optional, not required.

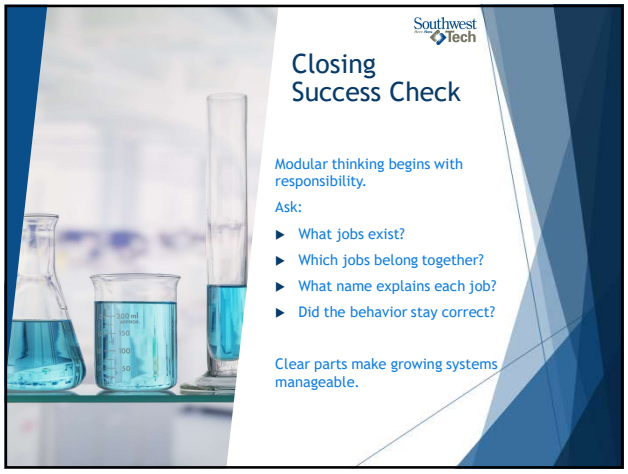
31


Southwest Tech
Evidence
the application still works
function names describe real jobs
responsibilities are separated intentionally
event handlers remain focused
repeated logic is reduced where reasonable
your reflection explains what became clearer
More functions alone do not prove modularity.

32


Southwest Tech
Modular Thinking Summary
1
find the jobs
2
group related jobs
3
name each responsibility
4
connect the parts
5
verify behavior
Clear parts make growing systems manageable.
Modular Thinking Summary

33


Southwest Tech
Closing Success Check
Modular thinking begins with responsibility.
Ask:
What jobs exist?
Which jobs belong together?
What name explains each job?
Did the behavior stay correct?
Clear parts make growing systems manageable.

34


Southwest Tech
Thank you!
Have Fun!

35